

Multicore Environment State Representation for Agent-Directed Test Generation

Bruno D. Miranda, Luiz M. V. Pereira, Márcio Castro, Luiz C. V. dos Santos

Federal University of Santa Catarina

Florianópolis, SC, Brazil

b.miranda@posgrad.ufsc.br; luiz.m.v.p@grad.ufsc.br; {marcio.castro, luiz.santos}@ufsc.br

Abstract—A crucial step in the design of multicore systems is to validate the interaction between cores. This involves test program generation and runtime analysis. We propose a novel reinforcement learning approach to directed test generation, where an agent induces a suite of programs, which are executed in a simulation environment for a multicore. It focuses on how to recover state information from raw observations of the environment such that the agent can learn from interaction how to improve coverage for any verification task. We evaluated our state representation for different verification tasks involving 16 and 32-core ARMv8 2-level MOESI designs.

Index Terms—Reinforcement learning, test generation.

I. INTRODUCTION

The verification of the interactions between cores in multicore systems requires the execution of concurrent test programs until coverage goals are met, and no errors are left in the design. Such interactions through shared memory must obey a *Memory Consistency Model* (MCM) [1], [2], which allows many valid behaviors for the same concurrent program, leading to multiple correct outcomes for the same input stimuli. The lack of a one-to-one golden model for the expected behavior makes the generation of self-checking tests very complex [3], [4].

A pragmatic alternative is to split verification into complementary tasks: a generator produces a suite of test programs, while an external checker analyzes whether or not the observed behavior during the execution of each program complies with the MCM [5]–[9]. This allows the generator to focus on coverage improvement, whereas the checker focuses on error diagnosis. This paper follows such strategy for test generation.

Various approaches to *Directed Test Generation* (DTG) have been reported for the target verification domain, such as Bayesian networks [10], Euler tour [11], [12], genetic programming [13], local search [14], and reinforcement learning [15].

Reinforcement learning (RL) involves the interaction between an *agent* and an *environment*. The agent receives a *reward* signal, and takes *actions* depending on the environment *state*. In many practical cases, the agent can only observe partial information about the state. The choice of proper *observations* depends on the target environment, and it impacts performance.

This paper proposes a novel reinforcement learning approach to DTG which focuses on proper state representation for

environments containing a *design under verification* (DUV) that is a multicore system. We show (1) what should be observed, (2) how observations should be represented, and (3) how they should be used to recover state information.

Firstly, we propose the notion of *witness* to execution instances of a concurrent program as a proper key to observing the response of a multicore environment to an action taken by an agent. To define a witness, we repurposed the well-known *reads-from* relation [16] for recovering state information. Despite its standard use for the synthesis of litmus tests [16], [17], that relation has never been cast before into a state representation for enabling RL agents to *direct* test generation.

Secondly, we show that test generation can be conveniently constrained to enable a provenly *unique representation* for every observable execution witness induced by a test suite.

Finally, we show how environment state information can be recovered from the bounded *history of action-observation pairs*.

This paper is organized as follows. Section II discusses our motivation in face of related work. Section III describes the proposed approach to DTG. Section IV formulates our proposal for suitable observation, proper representation, and state recovery. Section V reports our experimental evaluation, and Section VI draws overall conclusions.

II. MOTIVATION IN FACE OF RELATED WORK

Since model-based DTG [11], [12] is usually tied to protocol or architecture, data-driven approaches are preferred [10], [13]–[15]. One of them [10] dynamically redefines directives for a random test generation engine by means of statistical inference. That approach is based on modeling the relationship between coverage information and test directives as Bayesian networks. MCjammer [18] employs multiple agents in distinct core domains that generate concurrent memory access patterns. MCjammer is not a test program generator, but a data-driven verification environment that directly generates stimuli based on the coverage goals of multiple agents. The McVersi Test Generator (MTG) [13] relies on Genetic Programming, and it led to superior coverage and error discovery as compared to random test generation and litmus tests. MTG represents the state of the art for evolutionary DTG. The Reinforcement Learning Generator (RLG) [15] is the only RL approach to DTG reported so far for the target domain, and it obtained results competitive with MTG. Thus, MTG and RLG are qualified baselines for the evaluation of novel DTG approaches.

This work was financed in part by the Conselho Nacional de Desenvolvimento Científico e Tecnológico - Brasil (CNPq) - grants 307093/2021-2 (PQ), 308933/2020-6 (PQ) - and by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001.

Most DTG techniques are *coverage-directed* test generators [10], [13]–[15], where coverage is the only observation that is tracked from the DUV to direct the generation of new tests. However, coverage is not an injection, since the same fraction of the coverage goals may correspond to different subsets with same cardinality. Although the cumulative coverage captures the evolution of the verification process, it does not accurately track the actual trajectory of responses to the induced stimuli.

Thus, to be improved, DTG needs extra observations from the DUV, but such information is usually restricted to external checkers, because most generators rely on them for being able to focus on coverage improvement. Such extra observations should not be tied to MCM, coherence protocol or microarchitecture to avoid dependence on a specific verification task.

Such DUV-agnostic observations are naturally exploited by litmus tests [2]. Since litmus test generation [16], [17] cannot provide coverage guarantees [13], and coverage-directed generation is restricted due to limited observation, proper observations should be extracted from the DUV to guide novel approaches to DTG. This motivates the use of a sequence of domain-specific observations as a trajectory that can be used to recover the RL notion of environment state [19].

Successful applications of RL used to require handcrafted tuning [20] to each specific problem until the Deep Q-Network (DQN) approach [21] has shown that the combination of Q-learning [22], deep neural network, and experience replay [23] could handle a collection of different problems (video games) of the same application domain (Atari suite) without having to resort to problem-specific tuning. High performance was obtained with the same hyper-parameters, network architecture, and observations (raw video frames). Since then, several extensions to DQN have been reported [24].

This feature of recent agents is attractive to the verification of multicore systems, where different tasks are induced by a variety of coherence protocols, core counts, microarchitectures, and coverage metrics. To benefit from RL, the raw input for the agent should capture primal shared-memory behaviors, which are specified as *order relations* in formal frameworks for litmus test generation [16], [17] and MCM checking [6], [9]. Among them, the *reads-from* relation is particularly convenient, because it allows order to be inferred from observed values. The use of values for representing order was proposed as part of a profiling technique [25], where a signature encoded the order of basic blocks in an execution path of a sequential program. That idea inspired a technique for trace compaction [26], where signatures were used to capture the order of memory operations induced by concurrent programs. Such compaction technique inspired us to rely on signatures for environment *state recovery*, enabling the use of RL agents for the target verification domain.

III. THE PROPOSED APPROACH

A. Background in brief

The interaction between agent and environment occurs at discrete time steps $0, 1, 2, \dots, t$. At a given time step t , the agent receives some representation of the environment state, S_t , and it responds with some action A_t . In the next time

step, the environment sends to the agent a scalar reward signal R_{t+1} , and evolves to a new state S_{t+1} . Both R_{t+1} and S_{t+1} are determined as a consequence of action A_t . The agent learns from interaction how to take actions on the environment in order to maximize expected rewards. Thus, the interaction between agent and environment can be expressed as $\langle S_0, A_0, S_1, R_1, A_1, S_2, R_2, A_2, S_3, R_3, \dots, A_{t-1}, S_t, R_t \rangle$.

The interaction is usually formalized as a Markov decision process, where R_{t+1} and S_{t+1} can be defined only in terms of S_t and A_t . This means that a state must be able to completely capture all the interaction history that could be relevant for future decision making [19]. Such ideal property often does not hold in practice, because the agent observes only partial information about the environment state. Thus, some technique is required to approximate a Markov state from observations.

To handle partial observability, we do not assume that the environment emits the actual state, but it exposes some observation of signals that depends on that state. We also assume that rewards can be obtained from observations. Under these assumptions, the interaction can be expressed as an alternating sequence of actions A_{t-1} and observations O_t , as follows: $\langle A_0, O_1, A_1, O_2, A_2, O_3, \dots, A_{t-1}, O_t \rangle$.

B. Agent-based approach to DTG

Let P_t denote a concurrent *test* program, and let $P_1, P_2, \dots, P_t, \dots, P_T$ denote a *test suite*.

Typically, each concurrent program is executed a certain number of times to expose multiple *execution instances* that are likely to exhibit distinct shared-memory behavior. Let I denote a set of execution instances induced by a given program, which are obtained by running the program $|I|$ times, in such a way that each execution is differently perturbed due to the non-determinism of thread concurrency.

Coverage is quantified as a fraction of the coverage goals reached during the execution of the programs of a test suite. Let C_{t+1} be the cumulative coverage obtained by running I execution instances of every program of the sequence $P_1, P_2, \dots, P_t$.

DTG aims to synthesize a test suite $P_1, P_2, \dots, P_t, \dots, P_T$ that maximizes C_{T+1} . To cast DTG as an RL problem, we introduce the notion of *verification episode*, which comprises successive epochs, each defined by discrete time steps. Each epoch between steps t and $t + 1$ consists of three phases: test generation, multiple execution, and coverage evaluation, as follows. An action A_t induces the generation of a program P_t . Then P_t is executed $|I|$ times in the DUV. Finally, C_{t+1} is updated with the *individual* contribution of P_t to the cumulative coverage (i.e. the joint contribution of all its $|I|$ execution instances). This is used to define the reward obtained with P_t .

Definition 1: A *reward* is a function $\rho : \mathcal{R} \times \mathcal{R} \mapsto \mathcal{R}$ such that $R_{t+1} = \rho(C_{t+1}, C_t)$.

Fig. 1 illustrates our RL approach to DTG. The environment includes the DUV, a *test generation engine* (TGE), and a *coverage engine* (CVE), which computes the fraction of the coverage goals achieved by all instances of every program executed so far. Between environment and agent, three modules compute functions that convert actual environment information into the conventional interface expected by an RL agent.

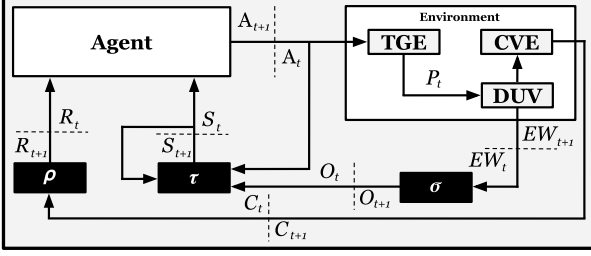


Fig. 1: A reinforcement learning approach to DTG

Dotted lines represent temporal barriers to indicate the evolution between successive time steps. At a given time step t , when the environment is in state S_t , the action A_t induces the generation of a test program P_t . Then multiple instances of that program are executed in the DUV. At time step $t+1$, monitors in the DUV provide a *witness* EW_{t+1} to those executions, and the coverage engine provides the cumulative coverage C_{t+1} .

The function ρ converts coverage information into a reward R_{t+1} , and the function σ converts witnessed information into an observation O_{t+1} . Both R_{t+1} and O_{t+1} are jointly obtained from the multiple executions of the test launched at step t .

The function τ computes a transition from the state S_t to the following state S_{t+1} for a given action-observation pair A_t, O_t .

The notion of execution witness and the functions σ and τ are the keys to our state representation, as follows.

IV. THE PROPOSED STATE REPRESENTATION

Let us define *what* should be observed (to properly capture the environment's activity) and *how* it should be encoded (to ensure a non-ambiguous, compact representation).

A. Choice of suitable observation and proper representation

Let $\mathcal{OP}$ be the set of memory operations of a program. Let S_j^i denote that store operation $S_j \in \mathcal{OP}$ executes in core i . Let L_m^k denote that load operation $L_m \in \mathcal{OP}$ executes in core k .

Each test may induce different execution instances of a concurrent program, which can be seen as distinct orderings of the memory operations of a concurrent program [27].

Definition 2: An *execution instance*, written $<$, is a total order on the set $\mathcal{OP}$.

An execution instance $<$ determines which value is observed by each load. Let Val^0 denote the initial value assigned to some location. Let $Val[S]$ denote the value written by some store S , and let $Val_{<}[L]$ be the value read by some load L .

An execution instance $<$ induces the following relation.

Definition 3: L and S are in the *reads-from* relation, written $L \text{ rf } S$, iff $Val_{<}[L^k] = Val[S^i] \neq Val^0$.

The outcome of a non-synchronized concurrent program may serve as witness to a given execution instance. First, let us define a witness from the perspective of some core i . Such local witness relies on the outcome induced by the set $\mathcal{L}^i$ of load operations belonging to the thread executed by that core.

Definition 4: A *core witness* to execution instance $<$ is $EW_{<}^i = \{v : v = Val_{<}[L] \wedge L \in \mathcal{L}^i\}$.

Note that a witness is determined by which store (if any) is observed by each load: if there exists some S such that $L \text{ rf } S$, then $Val_{<}[L] = Val[S]$, otherwise $Val_{<}[L] = Val^0$.

Now let us define a core witness induced by a set I of execution instances of the same test program.

Definition 5: A *core witness* to execution instances I is:

$$EW_I^i = \bigcup_{< \in I} EW_{<}^i.$$

Let us define an aggregation of the witnesses at each core.

Definition 6: A *multicore witness* to execution instances I is $EW_I = (EW_I^1, EW_I^2, \dots, EW_I^i, \dots, EW_I^p)$.

To properly encode execution witnesses, let us constrain store value assignment: different natural numbers must be assigned to distinct stores, and 0 is reserved for initial values, as follows.

Constraint 1: $Val[S_j] \in \mathcal{N}$ for all $S_j \in \mathcal{OP}$.

Constraint 2: $Val^0 = 0$ for all locations.

Constraint 3: $Val[S_j] \neq Val[S_{m \neq j}] > 0$.

Such constraints are commonly applied to test generation, because unique values serve as non-ambiguous identifiers for stores, thereby reducing the complexity of MCM checkers [6].

A straightforward way to satisfy Constraint 3 *would* be to assign the next natural number to the next synthesized store, during the generation of an *individual test* program. However, to rule out ambiguity when observing execution witnesses induced by *different* programs, let us define a unified assignment policy for all programs of a *test suite*. The key is to assign unique values to stores, independently of the number of stores chosen for each thread. Let θ denote an upper bound for the number of stores in a thread, and let S_j^i be the j -th store of a given thread i . The following implies Constraint 3.

Constraint 4: $Val[S_j^i] = \theta(i-1) + j$ and $1 \leq j \leq \theta$ for all tests.

To encode witnesses under those constraints, let $\mathcal{EW}^i = \{EW_{<}^i\}$ denote the witnesses induced by every possible execution instance $<$, and let $\mathcal{P}(\mathcal{EW}^i)$ be its power set.

Definition 7: The signature of a witness to execution instances I at core i is a function $sig : \mathcal{P}(\mathcal{EW}^i) \mapsto \mathcal{N}$ such that:

$$sig(EW_I^i) = \sum_{v \in EW_I^i} v$$

Let us prove that a signature encodes a core witness properly.

Lemma 1: Under Constraints 1, 2, and 4, sig is an injection.

Proof: Let us consider the witnesses $EW_I^i, EW_{I'}^i \in \mathcal{P}(\mathcal{EW}^i)$, and let us assume that they are different. $EW_I^i \neq EW_{I'}^i$ implies $I \neq I'$. Thus, I and I' must have each at least one execution instance, say $<$ and $<'$, which is not a member of the other, i.e.: $\{<\} \subseteq I \setminus I'$ and $\{<'\} \subseteq I' \setminus I$. Therefore, $EW_I^i \neq EW_{I'}^i$ implies $EW_{<}^i \neq EW_{<'}^i$ for some $<$ and $<'$. If $EW_{<}^i \neq EW_{<'}^i$ holds, there must exist at least one load $L \in \mathcal{L}^i$ such that $Val_{<}[L] = Val[S]$ and $Val_{<'}[L] = Val[S']$ or $Val_{<'}[L] = Val^0$. Since Constraint 4 implies Constraint 3, under Constraints 1, 2, and 3, we have $Val_{<}[L] \neq Val_{<'}[L]$. For convenience, let $U = \{u : u \in EW_{I \cap I'}^i\}$, $V = \{v : v \in EW_{I \setminus I'}^i\}$, and $V' = \{v' : v' \in EW_{I' \setminus I}^i\}$. From Definition 7, we have: $sig(EW_I^i) = \sum_{u \in U} u + \sum_{v \in V} v$ and $sig(EW_{I'}^i) = \sum_{u \in U} u + \sum_{v' \in V'} v'$. Since $Val_{<}[L] \neq Val_{<'}[L]$ holds for some L , it implies $\sum_{v \in V} v \neq \sum_{v' \in V'} v'$, which

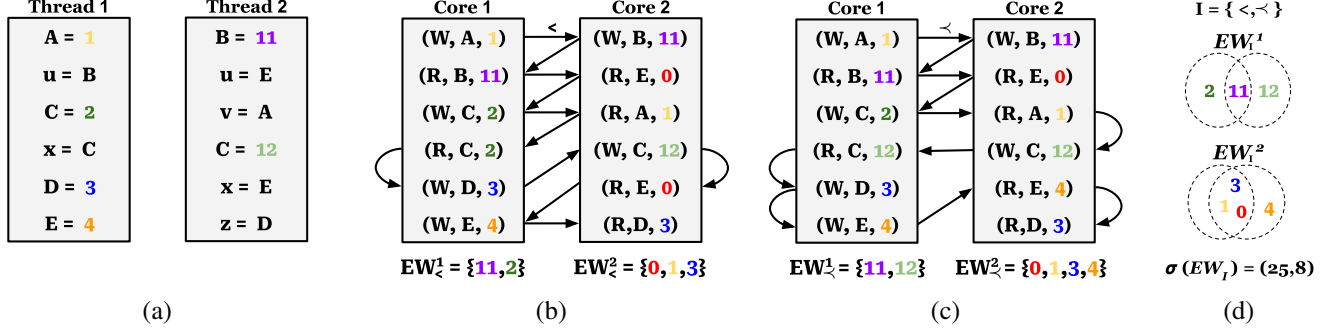


Fig. 2: An observation captured by a joint witness to execution instances $I = \{<, <-\}$, and its representation as a signature.

leads to distinct signatures. Since $EW_I^i \neq EW_{I'}^i$ implies $sig(EW_I^i) \neq sig(EW_{I'}^i)$, sig is an injection. ■

Let $\mathcal{EW} = \{EW_{<}\}$ be the set of *multicore* witnesses induced by every possible execution instance $<$.

Definition 8: The signature of a multicore witness to execution instances I is a function $\sigma : \mathcal{P}(\mathcal{EW}) \mapsto \mathcal{N}^p$ such that: $\sigma(EW_I) = (sig(EW_I^1), sig(EW_I^2), \dots, sig(EW_I^p))$.

Theorem 1: Under Constraints 1, 2, and 4, σ is an injection.

Proof: Let us consider the witnesses $EW_I, EW_{I'} \in \mathcal{P}(\mathcal{EW})$, and assume that they are different. If $EW_I \neq EW_{I'}$ holds, there must exist at least one core i such $EW_I^i \neq EW_{I'}^i$, which from Lemma 1 implies $sig(EW_I^i) \neq sig(EW_{I'}^i)$. From Definition 8, this is sufficient to conclude that $\sigma(EW_I) \neq \sigma(EW_{I'})$. Since $EW_I \neq EW_{I'}$ implies $\sigma(EW_I) \neq \sigma(EW_{I'})$, σ is an injection. ■

Theorem 1 guarantees that signatures are *unique* representations for observations of a multicore DUV.

B. Illustrative example

Fig. 2 illustrates the proposed representation for two execution witnesses of the same test program. Fig. 2(a) shows a test program with two threads, where capital letters denote shared memory locations, and small letters represent variables allocated in registers. Note that all values are natural numbers (Constraint 1), the locations are all initialized with zero (Constraint 2), and values written by stores are unique, greater than zero (Constraint 3), and obtained with $\theta = 10$ (Constraint 4).

Fig. 2(b) and Fig. 2(c) show distinct execution instances of that test program induced by different orders ($<$ and $<-$). In each core, a trace captures the sequence of memory accesses resulting from the execution of load and store operations. Each memory access is represented as a triple specifying the access type as write (W) or read (R), the location, and the value. Fig. 2(d) shows the joint witness and its signature.

From now on, when we need to timestamp a witness, instead of $EW_{I,t}$, we simply write EW_t (assuming an implicit set I).

C. Recovering of state information from observations

Given the sequence $\langle A_0, O_1, A_1, O_2, A_2, O_3, \dots, A_{t-1}, O_t \rangle$, the recurrent pattern A_{t-1}, O_t is the basis for the approximation for the notion of state, where the interaction history is bounded.

Definition 9: For a bound $h \geq 1$, a state at time step $t \geq h$ is $S_t = \langle A_{t-h}, O_{t-h+1}, \dots, A_{t-2}, O_{t-1}, A_{t-1}, O_t \rangle$.

A proper value for the bound h can be obtained empirically.

Let $\mathcal{S}$, $\mathcal{A}$, and $\mathcal{O}$ denote the sets of states, actions, and observations. A transition from state S_t to S_{t+1} can be computed by shifting the representation of S_t one pair to the left, and appending A_t, O_{t+1} , so as to keep history bounded [19].

Definition 10: A *transition* is a function $\tau : \mathcal{S} \times \mathcal{A} \times \mathcal{O} \mapsto \mathcal{S}$ such that $S_{t+1} = \langle A_{t-h+1}, O_{t-h+2}, \dots, A_{t-1}, O_t, A_t, O_{t+1} \rangle$.

We propose a state representation where $O_t = \sigma(EW_t)$.

V. EXPERIMENTAL EVALUATION

We built agent-based generators with the same set of actions. The *Random Agent Generator* (RAG) selects every action with the same probability. The *Deep-Q Generator* (DQG) relies on the DQN agent [24], and it has variants with distinct state representations, as shown at the top of Table I. RAG rules out learning effects (because its agent's policy is independent of state), serving as a baseline for evaluating the impact of state representation on learning, whether using general-purpose observations (coverage values in DQG⁺) or the proposed domain-specific observations (execution witnesses in DQG^{*}).

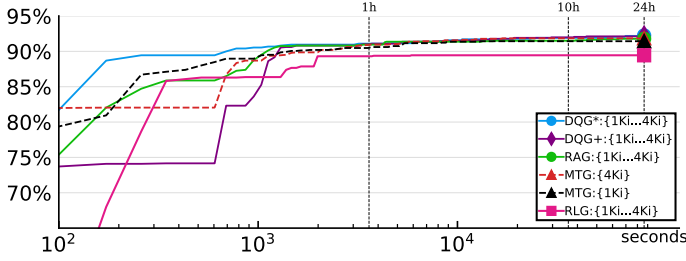
TABLE I: Generators used to evaluate our state representation

Generator	Agent	State Representation	
		Observation	Trajectory record
RAG	random	n.a.	n.a.
DQG ⁺	DQN [21]	C_t	action-observation history
DQG [*]		$\sigma(EW_t)$	action-observation history
RLG	[15]	C_t	t
MTG	none	n.a.	n.a.

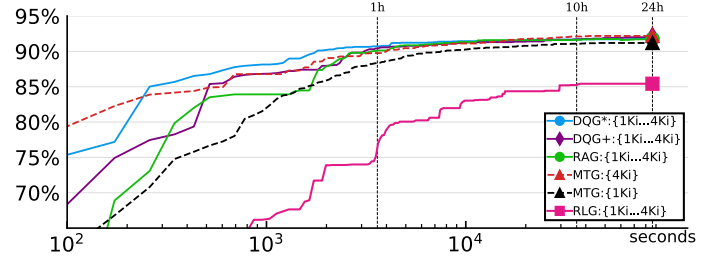
We compared RAG and DQG variants with directed test generators described in related works: the Reinforcement Learning Generator (RLG) [15], whose state representation is not based on action-observation history, and the McVersi Test Generator (MTG) [13], which does not rely on an agent, but on evolutionary directed test generation, as shown at the bottom of Table I.

We evaluated each generator when performing different *verification tasks* targeting ARMv8 DUVs. Each task is obtained as a combination of distinct core counts (16 and 32) with different coverage metrics (structural transition coverage, as defined in [13], and functional transition coverage, as defined in [14]).

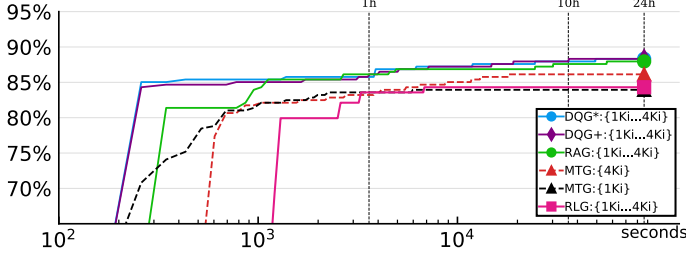
We relied on gem5's infrastructure [28] for simulation and design representation (we chose *O3*, *Ruby*, and *simple* for CPU, memory, and network models, respectively). For the DUVs, we



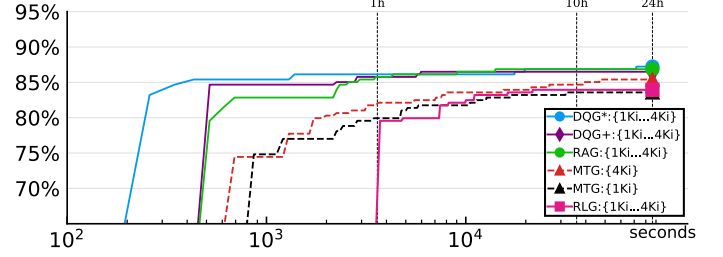
(a) Task 1: 16-core, functional metric



(b) Task 2: 32-core, functional metric



(c) Task 3: 16-core, structural metric



(d) Task 4: 32-core, structural metric

Fig. 3: Median (cumulative) coverage evolution for 24-hour verification episodes

adopted 64KiB (4-way) private L1 caches and a 4MiB (16-way) shared L2 cache, all with same block size (64 bytes), under a 2-level MOESI directory-based coherence protocol.

We ran experiments under the same range constraints on the number of operations ($n_{min} = 1Ki$, $n_{max} = 4Ki$) and shared locations ($s_{min} = 4$, $s_{max} = 128$). The allowed values within those ranges had to comply with generator’s differences¹.

RAG, DQG, and RLG can generate programs with *variable* numbers of operations in the same test suite, but MTG does not: all programs must have a *fixed* number of operations. That is why we report individual MTG results for range extremes.

DQG employs a DQN agent, which uses a *Q-network* for non-linear approximation of the state-action value function. We adopted a feed-forward Q-network containing two hidden layers with 256 neurons each (ReLU and tanh were used for activation in successive layers). At each discrete time step t , the agent maximizes the expected discounted return $G_t = \sum_{t'=t+1}^T \gamma^{t'-(t+1)} R_{t'}$. The values chosen for the hyper-parameters were: discount rate $\gamma = 0.9$, learning rate $\eta = 0.025$, batch size $\beta = 32$, and the probability of taking a random action was linearly scaled down from $\epsilon = 0.9$ (when the episode starts) down to $\epsilon = 0.05$ (when it reaches one hour). We adopted $h = 4$ as history bound and the reward function defined by Equation 1.

$$R_{t+1} = \rho(C_{t+1}, C_t) = \begin{cases} +1 & \text{if } C_{t+1} > C_t \\ -1 & \text{otherwise} \end{cases} \quad (1)$$

The number of execution instances of each test was set to five, and was induced as suggested in [13]. In each verification episode, every test was run five times under different simulation states (not related to the test itself) in such a way that the distinct executions of the same test were all perturbed differently.

¹MTG requires the static definition of a *single* address-space constraint, which is called *test memory size* (TM). To accommodate such difference, we report MTG results under the (best) static constraint defined in [13], i.e. TM=8KiB. We preserved all genetic parameters as they were originally set.

Runtimes were measured on a Linux Server (Intel Xeon E5-2640 v4, 2.4 GHz, with 128GB of main memory).

A. Coverage evolution with time

We performed independent 24-hour verification episodes for each task. We launched each generator five times with distinct random seeds, and reported median coverage values. For DQG and RLG, a new set of random weights was used to initialize their neural networks at each new launching.

Fig. 3 shows the coverage evolution for different tasks. RLG led to the slowest coverage evolution. The state representations used by RLG and DQG⁺ both rely on coverage values as generic observations. However, RLG does not take the history of action-observation pairs into account to approximate the environment state as DQG⁺ does. This explains why RLG exhibits limited coverage evolution as compared to DQG⁺. Note that DQG* was superior to RAG for all tasks, while DQG+ was mostly superior to RAG, except for Task 1. This indicates that the history of action-observation pairs *did* enable learning, which is more prominent when our state representation is used.

DQG* was superior to DQG⁺, despite environment changes from one task to another. Thus, DQG* observations capture relevant information about the environment *independently of task*, i.e. they are properly tailored to the target *domain*.

As compared to related works, DQG* had the fastest coverage evolution for all tasks with exactly the same agent. Under the proposed state representation, the agent was able to learn on-line how to handle different tasks from “raw” observations of the environment, i.e. observations that were *not* represented in terms of specialized features handcrafted for any given task. Thus, a learning agent can make a directed test generator *effective* and *reusable* for distinct tasks of the same domain.

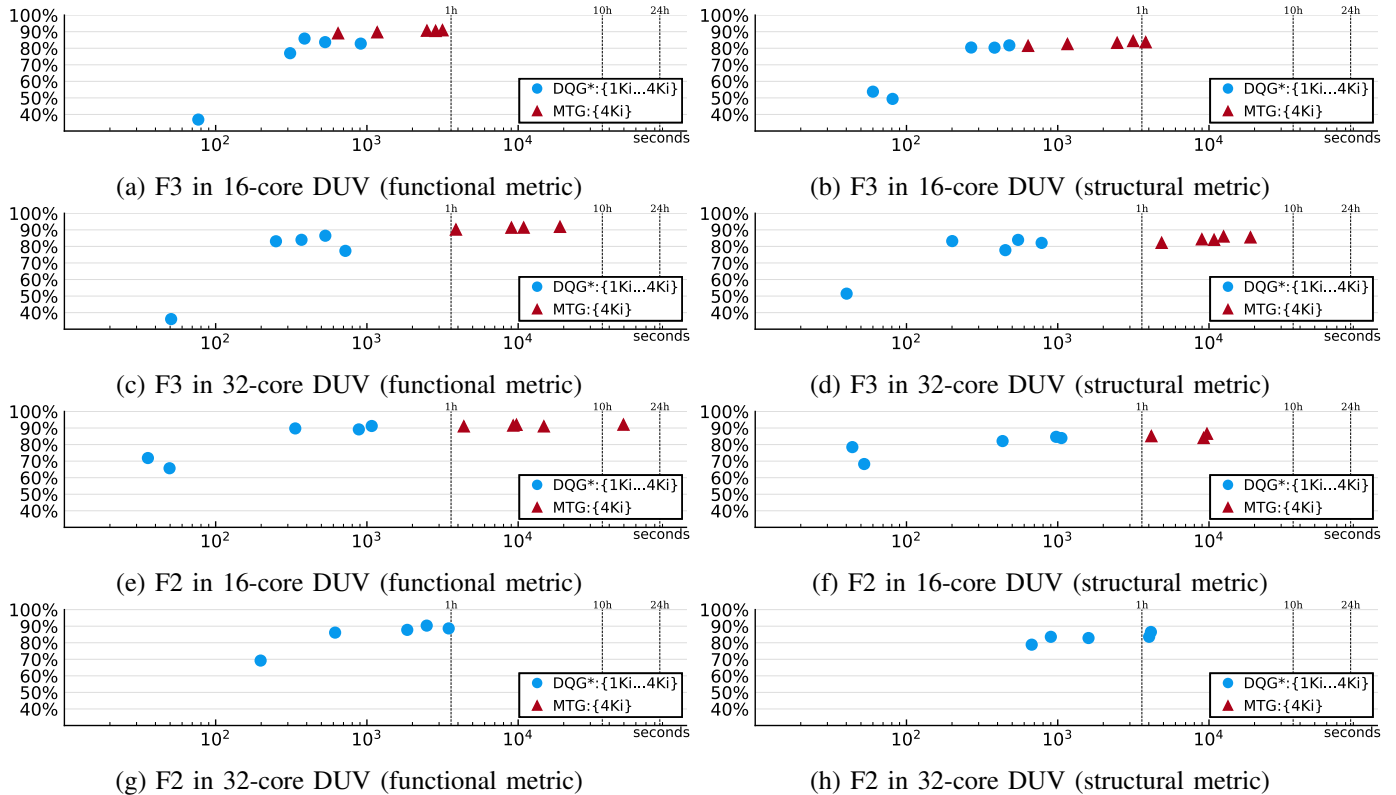


Fig. 4: Coverage-time error discovery constellations for faulty designs

B. Impact on error discovery

We evaluated the impact of our state representation on the runtime required to expose errors in faulty designs. For the experiments, we selected challenging errors described in the literature, namely F2 and F3 [9]. We modified our DUVs to obtain faulty designs, each containing a single error. Since coverage results have shown that DQG* dominates DQG+ and MTG dominates RLG, we focused on comparing the best DQG version (DQG*) with the best-performing related work (MTG).

TABLE II: Median error discovery times (in seconds)

Error	Cores	Functional metric		Structural metric	
		DQG*	MTG	DQG*	MTG
F3	16	386	2493	267	2470
	32	369	9948	450	10802
F2	16	335	9744	431	9216
	32	1848	-	1597	-

We launched five verification episodes with distinct random seeds. We measured the runtime until the error was discovered and the coverage at the time of discovery. Then we plotted a constellation of coverage-time points for each generator. Fig. 4 shows constellation plots for faulty designs with a single error under a given coverage metric. DQG* exposed every error in all 24-hour episodes, but MTG not always did. For a 32-core DUV, MTG did *not* expose F2 in *any* episode. When MTG exposed an error, it took much longer, as shown in Table II.

DQG* led to the fastest error diagnosis in all scenarios, it exposed errors with lower coverage (on average), and its discovery times were less sensitive to the increase in core count. This indicates that the proposed state representation leads to high-quality tests, quite independently of coverage metric.

Higher-quality tests can be obtained when an agent directs generation instead of an evolutionary algorithm, mainly because the former is farsighted, while the latter is myopic. The former takes decisions based on expected returns over the long run. The latter takes them based on immediate returns (i.e. the best for the current population, regardless of next generations).

VI. CONCLUSIONS

This paper proposed a novel reinforcement learning approach to directed test generation for a crucial step of the design of a multicore system: the verification of the interaction between cores through shared memory. To be effective, that approach should rely on a proper representation for the state of a multicore environment, which requires the tracking of the history of action-observation pairs, and proper tailoring of observations to the target domain. This allows the agent-based generator to learn online how to be effective and reusable for *different* tasks of the *same* domain. Results have shown that, when encoded as unique signatures, observations capturing the *reads-from* relation led to faster coverage evolution and faster error diagnosis, as compared to related work. This makes our approach to directed test generation a promising alternative to conventional techniques that are not based on agents.

REFERENCES

- [1] S. V. Adve and K. Gharachorloo, "Shared memory consistency models: a tutorial," *Computer*, vol. 29, no. 12, pp. 66–76, 12 1996.
- [2] V. Nagarajan, D. Sorin, M. Hill, and D. Wood, *A Primer on Memory Consistency and Cache Coherence, 2nd Edition*, ser. Synthesis Lectures on Computer Architecture. Morgan & Claypool Publishers, Feb. 2020.
- [3] A. Adir and G. Shurek, "Generating concurrent test-programs with collisions for multi-processor verification," in *7th IEEE Int. High-Level Design Validation and Test Workshop*, 2002, pp. 77–82.
- [4] A. Adir, E. Almog, L. Fournier, E. Marcus, M. Rimón, M. Vinov, and A. Ziv, "Genesys-Pro: innovations in test program generation for functional processor verification," *IEEE Design Test of Computers*, vol. 21, no. 2, pp. 84–93, Mar 2004.
- [5] A. Saha, N. Malik, B. O’Krafa, J. Lin, R. Raghavan, and U. Shamsi, "A simulation-based approach to architectural verification of multiprocessor systems," in *Proceedings International Phoenix Conference on Computers and Communications*, 1995, pp. 34–37.
- [6] C. Manovit and S. Hangal, "Completely verifying memory consistency of test program executions," in *IEEE Int. Symposium on High-Performance Computer Architecture (HPCA)*, 2006, pp. 166–175.
- [7] A. Roy, S. Zeisset, C. J. Fleckenstein, and J. C. Huang, *Fast and Generalized Polynomial Time Memory Consistency Verification*, ser. Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2006, vol. 4144, pp. 503–516.
- [8] O. Shacham, M. Wachs, A. Solomatnikov, A. Firoozshahian, S. Richardson, and M. Horowitz, "Verification of chip multiprocessor memory systems using a relaxed scoreboard," in *IEEE/ACM International Symposium on Microarchitecture (MICRO 41)*, 2008, pp. 294–305.
- [9] M. Graf, G. A. G. Andrade, and L. C. V. dos Santos, "Evecheck: An event-driven, scalable algorithm for coherent shared memory verification," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 2, pp. 683–696, 2023.
- [10] S. Fine and A. Ziv, "Coverage Directed Test Generation for Functional Verification Using Bayesian Networks," in *ACM Design Automation Conference (DAC)*, 2003, pp. 286–291.
- [11] X. Qin and P. Mishra, "Automated generation of directed tests for transition coverage in cache coherence protocols," in *Design, Automation, and Test in Europe (DATE)*, 2012, pp. 3–8.
- [12] Y. Lyu, X. Qin, M. M. Chen, and P. Mishra, "Directed test generation for validation of cache coherence protocols," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 38, no. 1, pp. 163–176, Jan 2019.
- [13] M. Elver and V. Nagarajan, "McVerSi: A test generation framework for fast memory consistency verification in simulation," in *IEEE Int. Symp. on High Perform. Computer Architecture (HPCA)*, 2016, pp. 618–630.
- [14] G. A. G. Andrade, M. Graf, N. Pfeifer, and L. C. V. dos Santos, "A directed test generator for shared-memory verification of multicore chip designs," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 12, pp. 5295–5303, 2020.
- [15] N. Pfeifer, B. V. Zimpel, G. A. G. Andrade, and L. C. V. dos Santos, "A Reinforcement Learning Approach to Directed Test Generation for Shared Memory Verification," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 3 2020, pp. 538–543.
- [16] J. Alglave, L. Marangé, S. Sarkar, and P. Sewell, "Fences in weak memory models," in *Computer Aided Verification*, vol. 6174, Dec 2010, pp. 258–272.
- [17] D. Lustig, A. Wright, A. Papakonstantinou, and O. Giroux, "Automated Synthesis of Comprehensive Memory Model Litmus Test Suites," in *ACM Int. Conf. on Arch. Support for Progr. Lang. and Op. Systems (ASPLOS)*, 2017, pp. 661–675.
- [18] I. Wagner and V. Bertacco, "MCjammer: Adaptive Verification for Multi-core Designs," in *Design, Automation, and Test in Europe (DATE)*, 2008, pp. 670–675.
- [19] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: A Bradford Book, 2018.
- [20] G. Tesauro, "Temporal difference learning and td-gammon," *Commun. ACM*, vol. 38, no. 3, p. 58–68, Mar. 1995. [Online]. Available: <https://doi.org/10.1145/203330.203343>
- [21] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing atari with deep reinforcement learning," in *NIPS Deep Learning Workshop 2013*, 2013.
- [22] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, May 1992. [Online]. Available: <http://www.springerlink.com/content/p120815501618373/fulltext.pdf>
- [23] L.-J. Lin, "Reinforcement learning for robots using neural networks," Ph.D. dissertation, Computer Science, USA, 1992, uMI Order No. GAX93-22750.
- [24] M. Hessel, J. Modayil, H. van Hasselt, T. Schaul, G. Ostrovski, W. Dabney, D. Horgan, B. Piot, M. Azar, and D. Silver, "Rainbow: Combining improvements in deep reinforcement learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.
- [25] T. Ball and J. Larus, "Efficient path profiling," in *Proceedings of the 29th Annual IEEE/ACM International Symposium on Microarchitecture. MICRO 29*, 1996, pp. 46–57.
- [26] D. Lee and V. Bertacco, "Mtracecheck: Validating non-deterministic behavior of memory consistency models in post-silicon validation," in *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 201–213.
- [27] K. Gharachorloo, "Memory consistency models for shared-memory multiprocessors," Ph.D. dissertation, Stanford University, 1995.
- [28] N. Binkert et al., "The gem5 simulator," *SIGARCH Comput. Archit. News*, vol. 39, no. 2, pp. 1–7, Aug 2011.

Guarder: A Stable and Lightweight Reconfigurable RRAM-based PIM Accelerator for DNN IP Protection

Ning Lin^{1,2,3,*}, Yi Li^{1,*}, Jiankun Li^{1,*}, Jichang Yang^{1,*}, Yangu He¹, Yukui Luo⁵, Dashan Shang², Xiaoming Chen^{4,†}, Xiaojuan Qi^{1,†}, Zhongrui Wang^{3,†}

¹Department of Electrical and Electronic Engineering, The University of Hong Kong, Hong Kong, China; ²State Key Lab of Fabrication Technologies for Integrated Circuits and Key Laboratory of Microelectronic Devices and Integrated Technology, Institute of Microelectronics of the Chinese Academy of Sciences (CAS), Beijing, China; ³School of Microelectronics, Southern University of Science and Technology, Shenzhen, China; ⁴Institute of Computing Technology of CAS, Beijing, China; ⁵Binghamton University, State University of New York, Binghamton, NY, USA.

*Equal contribution to this work. †Corresponding author: wangzr@sustech.edu.cn; chenxiaoming@ict.ac.cn; xjq@eee.hku.hk.

Abstract—Deploying deep neural networks (DNNs) on conventional digital edge devices faces significant challenges due to high energy consumption. A promising solution is the processing-in-memory (PIM) architecture with resistive random-access memory (RRAM), but RRAM-based systems suffer from imprecise weights due to programming stochasticity and cannot effectively utilize conventional weight encryption/decryption intellectual property (IP) protection schemes. To address these issues, we propose a novel software-hardware co-design Guarder. On the hardware side, we introduce 3T2R cells to achieve reliable multiply-accumulate (MAC) operations and use reconfigurable inverter operating voltages to encode keys for encrypting DNNs on RRAM. On the software side, we implement a contrastive training method that ensures high model accuracy on authorized chips while degrading performance on unauthorized ones. This approach protects DNN IP with minimal hardware overhead while significantly mitigating the effects of RRAM programming stochasticity. Extensive experiments on tasks such as image classification (using MLP, ResNet, and ViT), segmentation (using SegFormer), and image generation (using DiT) validate the effectiveness of our method. The proposed contrastive training ensures negligible performance degradation on authorized chips, while performance on unauthorized chips drops to random guessing or generation. Compared to traditional RRAM accelerators, the 3T2R-based accelerator achieves a $1.41\times$ reduction in area overhead and a $2.28\times$ reduction in energy consumption.

Index Terms—IP Protection, PIM, Reconfiguration, Contrastive training.

I. INTRODUCTION

Deep neural network (DNN) models, particularly vision foundation models such as Vision Transformers (ViTs) [1] and generative models such as diffusion models [2], are widely deployed in edge applications like personal healthcare [3], autonomous driving and safety [4], [5], and entertainment [6]. With increasing concerns about data privacy and security [7], edge deployment of these models offers significant benefits by protecting user privacy, eliminating data communication between devices and the cloud, and enabling offline services. However, implementing these advanced DNNs on traditional von Neumann architecture edge devices is challenging due to excessive energy consumption and latency.

A promising solution to this challenge is the use of processing-in-memory (PIM) architectures with non-volatile memories (NVMs), such as RRAM. By integrating compu-

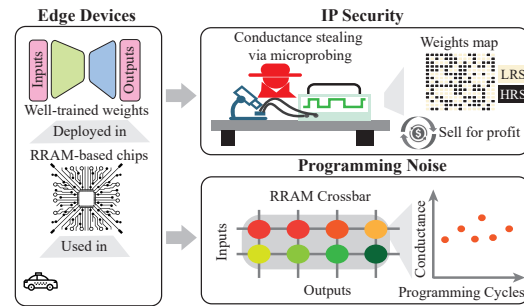


Fig. 1. Two main challenges of using RRAM-based PIM for DNN acceleration on edge devices. (1) Non-volatility makes model parameters prone to theft; (2) Programming noise can impact computation results.

tation, memory, and storage within a single physical unit, RRAM-based PIM significantly reduces the energy overhead associated with data movement and improves parallelism. Despite its advantages, RRAM-based PIM faces two critical challenges: (1) substantial stochasticity in RRAM programming leads to imprecise weights [8], and (2) persistent data in RRAM devices makes them vulnerable to security breaches. Attackers can physically probe the memory to extract DNN weights (Fig. 1) [9], [10], and traditional encryption methods [11], [12] are difficult to implement due to both PIM architecture and programming stochasticity of RRAM.

To address these issues, we propose a novel software-hardware co-design framework. Our approach introduces a 3T2R PIM array with reconfigurable inverter operating voltage on the hardware side, and a contrastive training method on the software side. Together, these innovations enhance computational stability and security while creating a significant performance gap between authorized and unauthorized chips. Specifically, the contributions of this work include:

- **A secure and robust IP protection method:** We propose a software-hardware co-design Guarder that protects DNN IP on RRAM with minimal encryption overhead, while also effectively mitigating the impact of RRAM programming stochasticity.
- **Hardware innovations:** We introduce 3T2R cells to construct crossbar arrays. The inverters in the 3T2R cells eliminate weight imprecision due to programming stochasticity, and the reconfigurable inverter operating

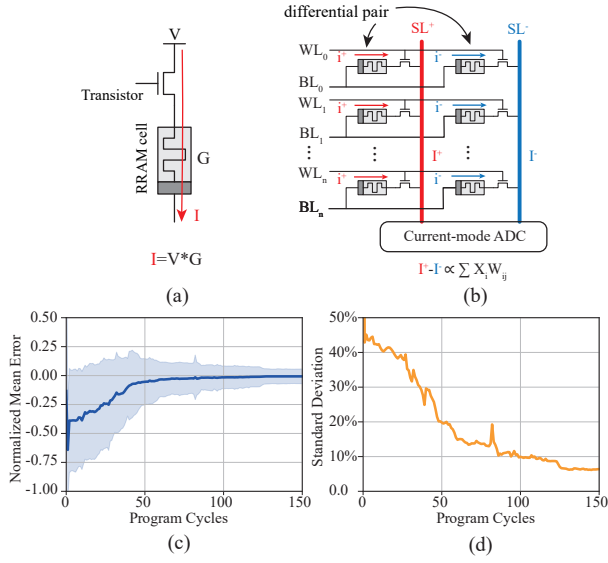


Fig. 2. RRAM-based PIM MAC principle (a) (b) and RRAM programming stochasticity (c) (d).

voltages serve as keys to encrypt weights, preventing attackers from exploiting the deployed weights.

- **Software advancements:** We develop a contrastive training method that ensures high model accuracy on authorized chips while degrading performance on unauthorized chips (e.g., general-purpose GPUs, CPUs) to near-random guessing levels.
- **Extensive validation:** We validated our approach on tasks like image classification, segmentation, and generation. Results show stable performance on authorized chips and significant degradation on unauthorized ones, ensuring effective model protection. The 3T2R-based design reduces chip area by $1.41\times$ and energy consumption by $2.28\times$ compared to previous RRAM designs, and withstands up to 30% RRAM programming stochasticity.

II. PRELIMINARIES

A. RRAM-based DNN Accelerators

Computational Principle. RRAM is a promising non-volatile PIM technology, offering high integration density, low energy consumption, and reduced fabrication costs [13]–[16]. As shown in Fig. 2 (a), a single RRAM device functions as a variable resistor. When paired with a transistor, it forms the classic 1T1R structure [17], generating output current based on Ohm’s law. Fig. 2 (b) illustrates that arranging these 1T1R cells in a 2D crossbar structure allows the currents from different 1T1R cells on the same source line (SL) to accumulate according to Kirchhoff’s current law, or weighted-current summation (WCS). A differential pair formed by two adjacent SL vectors of 1T1R units, when processed through an ADC [18], can represent both positive and negative values, corresponding to the weights in a DNN and their MAC operations with the input. However, WCS has inherent limitations: (1) Current sensing leads to high static power consumption. (2) The accumulated MAC result reflects individual RRAM

programming errors [19]–[21]. (3) It is incompatible with traditional weight storage encryption techniques [11], [12].

RRAM Device Characteristics. RRAM cells exhibit significant device-to-device and cycle-to-cycle programming stochasticity due to the underlying electrochemical reactions. Typically, a large number of programming cycles are required to approach the target resistance. Specifically, we programmed 250 real RRAM devices fabricated under 180 nm technology and analyzed the normalized mean error $((G_p - G_t)/G_t)$, where G_p is the programmed conductance and G_t is the target conductance) and the standard deviation of the programming errors, as shown in Fig. 2 (c) and (d). The results indicate that when the programming cycles are fewer than 50, the mean error and standard deviation are relatively large, exceeding -0.2 and 20%, respectively. As the number of programming cycles increases, the programming error gradually decreases. This yields large time and energy overheads.

B. IP Protection on RRAM

Prior works addressing IP protection on RRAM primarily employ encryption methods and circuit obfuscation, yet these approaches seldom consider the stochastic nature of RRAM.

Encryption. Cai *et al.* [11], [12] proposed a lightweight method known as sparse fast gradient encryption. This technique involves adding perturbations to the conductance values of the RRAM during encryption, which can be removed during decryption. However, it overlooks the inherent stochasticity of RRAM. Due to programming stochasticity, RRAMs often require multiple programming cycles to accurately write a weight, the energy consumption and latency adversely affect the DNN inference.

Circuit obfuscation. Zou *et al.* [22] proposed obfuscating the connections between positive and negative crossbars. During inference, the obfuscation module must be configured with the correct keys; without these keys, the stolen DNNs becomes non-functional. This approach, however, requires complex peripheral circuits, resulting in substantial area and energy overhead, which is particularly problematic for edge devices. Furthermore, this method does not address the stochasticity of RRAM.

III. GUARDER FRAMEWORK

A. Threat Model

In our threat model, we focus on real-time security-sensitive applications, such as segmentation in self-driving systems or drones, and generative models on mobile devices. We consider the victim to be the model provider and hardware provider (e.g., Nvidia’s Jetson NANO) [23]. The attacker is assumed to be the client user operating on the edge side. The model vendor must protect their valuable model IP before deploying the model on edge devices.

Attacker’s Capabilities. We assume that the attacker aims to extract a well-trained DNN model. With physical access to RRAM-based PIM accelerators, the attacker can read memory to extract DNN weights and deduce the architecture [9], [24],

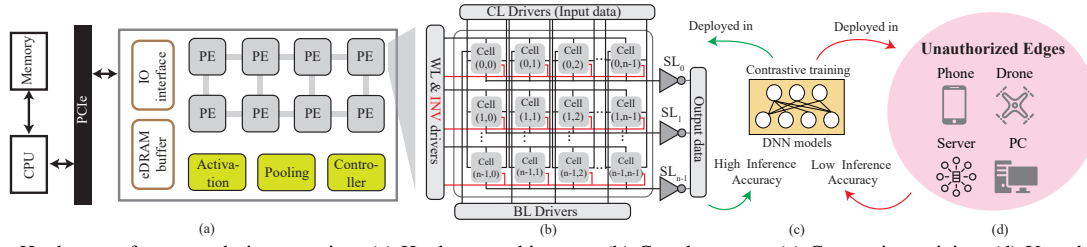


Fig. 3. Hardware-software co-design overview. (a) Hardware architecture; (b) Crossbar array; (c) Contrastive training; (d) Unauthorized edges.

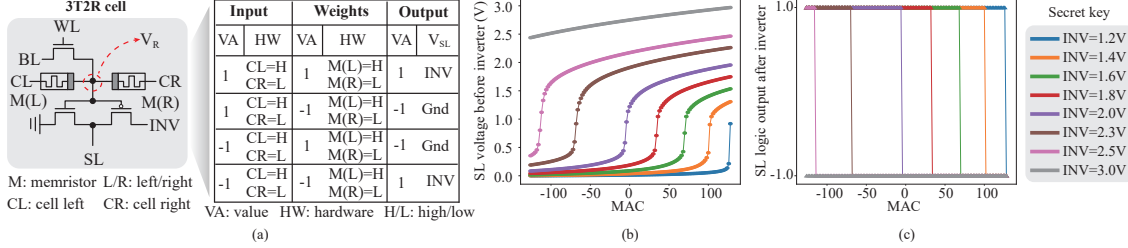


Fig. 4. (a) Noise-robust 3T2R cell design; (b-c) SL voltages before inverter and SL logic output after inverter as a function of MAC result at reconfigurable INV voltages, respectively.

[25]. Such extracted information could be exploited for profit, for instance, by reselling the stolen model.

Attacker's Limitations. Following existing literature (e.g., [26], [27]), we assume that attackers lack access to the vendor's proprietary training data, which is crucial for retraining the stolen DNN parameters. This training data is typically private, safeguarded by data protection laws such as the EU GDPR [28], and holds significant commercial value, making it inaccessible to the attacker.

B. Overview

To address the stochastic nature of RRAM and the challenges of IP protection, we propose a novel hardware-software co-design, as illustrated in Fig. 3. On the hardware side, the 3T2R design has been experimentally validated to withstand up to 30% of the RRAM programming stochasticity. Furthermore, the reconfigurable array effectively prevents attackers from utilizing the functions of the weights deployed on the RRAM after they have been stolen. On the algorithm side, we propose a contrastive training method, which ensures that the model achieves high accuracy and performance on authorized chips, while the accuracy on unauthorized edge chips is reduced to near-random guess levels.

C. Hardware Design

The RRAM-based PIM architecture is depicted in Fig. 3 (a). The top level of the chip contains Processing Elements (PEs), an eDRAM buffer, and neural-functional peripheries, including activation functions and pooling, and controller. Different layers of DNNs are typically mapped to different PE units. Additionally, when the length of the unrolled weights in the same layer exceeds the size of the crossbar, the weights are split and mapped onto different PEs.

Noise-Robust 3T2R Cell. Inspired by the voltage division cell design in [19]–[21], we propose an innovative 3T2R cell (Fig. 4 (a)). This cell integrates two RRAMs in complementary states, an inverter, and a programming transistor. The output of the RRAM voltage divider (referred to as V_R in Fig. 4 (a))

is binarized by the inverter, effectively performing an AND operation between the input (referred to as CL/CR in Fig. 4 (a)) and the stored conductance. The binary inverter within the 3T2R cell also mitigates RRAM programming stochasticity impact on the cell output.

More specifically, the truth table illustrates the relationship between different inputs and weights and their corresponding outputs. For instance, when the input value is logic '1', the CL voltage is high, while the CR voltage is low. When the weight is logic '1', the corresponding left RRAM (M(L)) is in a high resistance state, and the right RRAM (M(R)) is in a low resistance state. The product of the input and output can be measured at the SL, where a measured $V_{SL} = INV$ corresponds to logic '1'.

Security-Reconfigured PE Array. Within a single crossbar, binary weights are stored in the array. Input vectors are CL voltages and the MAC results are read out in parallel from the SL invertors. The WL, BL, and INV control RRAM selection, programming and encryption, respectively, as shown in Fig. 3 (b).

We reconfigure a chip with a crossbar size of 128×128 . Cells on the same SL share the same inverter supply voltage (INV) (red line in Fig. 3(b)). Different SLs have different INV voltages. Each combination of INV voltages across all SLs is a key for encryption. This is because, for a given SL, as the INV voltage increases from 1.2V to 3.0V, the SL voltage (before inverter) rises (Fig. 4(b)) accordingly. Consequently, the MAC threshold where the SL logic output flips right shifts. For instance, as shown in Fig. 4(c), when the INV is 2.0V, the MAC threshold is 0. When the INV reduces to 1.6V, the SL logic output is '-1' only if the SL MAC result is greater than threshold 50; otherwise, it is '1'.

As a result, the 3T2R array performs both MAC operations and reconfigurable binarizations, equivalent to a binary neural network with tunable sign activation.

RRAM-friendly Encryption. To illustrate DNN protection, we use a single MLP layer as an example. Fig. 5 (a) shows

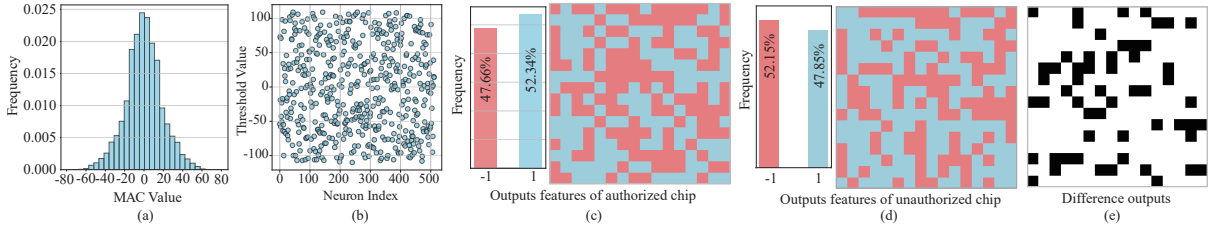


Fig. 5. Encryption comparisons. (a) Histogram of MAC results; (b) Threshold values for different output neurons; (c) Output features of the authorized chip; (d) Output features of the unauthorized chip; (e) Feature differences.

the output features the MLP layer, which follows a Gaussian distribution centered around 0. We first implement this layer on the authorized chip, with a secret key k illustrated in Fig. 5 (b). Here the key refers to the collection of MAC thresholds of different SLs, ranging from -110 to 110, which are physically implemented using different INV voltages on different SLs. The encrypted authorized chip's output features in Fig. 5 (c) show '-1' and '1' counts of 47.66% and 52.34%, respectively. In contrast, the unauthorized chip with a default key (i.e. 0) produces features shown in Fig. 5 (d), with '-1' and '1' counts of 52.15% and 47.85%. The feature map differences are revealed in Fig. 5 (e). Thus, the authorized chip's output significantly differs from the unauthorized one, thank to lightweight and fast INV settings without dedicated encryption/decryption modules.

D. Software Design

Naive Training. One intuitive method is to train the weights with the authorized chip by minimizing corresponding loss function. However, this approach cannot ensure that the unauthorized chip used by an attacker will be unusable.

Specifically, as illustrated in Fig. 6 (a), the performance gaps across various models, including shallow MLP models on MNIST dataset and deeper architectures like ResNet and ViT, on CIFAR10 dataset classification are approximately 13%, 10%, and 2%, respectively. These relatively minor differences are insufficient.

Contrastive Training. To increase this performance gap, we propose a novel contrastive training method. This approach aims to train a DNN that excels when running on the authorized chip with a secret key, but performs poorly on the unauthorized chip with the default key (i.e., INV = 2.0V, corresponding to SL MAC threshold = 0).

Let X and Y^t denote the input data and true labels of the dataset, respectively. f represents the DNN model, and $\mathcal{L}$ is the loss function (cross-entropy or mean squared error). Our contrastive training can be formulated as,

$$\begin{aligned} \min \mathcal{L} &= \min_X \mathbb{E} \mathcal{L}(f(k, X), Y^t) + \lambda \times \max_X \mathbb{E} \mathcal{L}(f(0, X), Y^t) \\ &\simeq \min_X (\mathbb{E} \mathcal{L}(f(k, X), Y^t) - \lambda \times \mathbb{E} \mathcal{L}(f(0, X), Y^t)) \end{aligned} \quad (1)$$

Here, $\mathbb{E}$ represents the expectation operator, and λ is a hyperparameter controlling the importance of accuracy gap between the authorized and unauthorized chips. k represents the INV voltage set of our chip.

This formulation aims to minimize the loss with the secret key k , while maximizing the loss with the default key

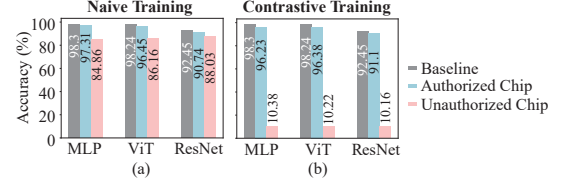


Fig. 6. The performance comparison of (a) the naive training and (b) contrastive training on classification models.

0, enhancing the performance gap between authorized and unauthorized scenarios.

To prevent training collapse due to a negative total loss $\mathcal{L}$, we redefine the hyperparameter λ using magnitude normalization. Specifically, we compute the logarithmic difference between $\mathcal{L}(f(0, X), Y^t)$ and $\mathcal{L}(f(k, X), Y^t)$, and adjust it with α ,

$$\lambda = \alpha \times 10^{\left(\log_{10} \frac{\mathcal{L}(f(0, X), Y^t)}{\mathcal{L}(f(k, X), Y^t)}\right)} \quad (2)$$

This redefinition ensures the magnitude difference between $\mathcal{L}(f(0, X), Y^t)$ and $\mathcal{L}(f(k, X), Y^t)$ is appropriately adjusted, maintaining the primary task of enhancing the accuracy of the authorized chip.

IV. EVALUATION

A. Experimental Setup

Software Benchmarks. Evaluations were conducted on tasks including classification, detection, segmentation, and image generation. For classification, an MLP with four fully connected layers was validated on the MNIST dataset [29], while deeper models such as ResNet [30] and Vision Transformer (ViT) [1] were validated on the CIFAR-10 dataset [31]. Segmentation tasks utilized Segformer B0 model on the VOC dataset. For image generation, the Diffusion Transformer (DiT) B/2 [2] was validated on the Animal Faces-HQ (AFHQ) dataset [32]. In these models, the weights of the layers requiring encryption were quantized to binary weights before being mapped to 3T2R array. All models were trained using the PyTorch framework.

Hardware Accelerators. The Cadence SPICE model of the 3T2R array was simulated using UMC's 180 nm technology node. The RRAM in the 3T2R cell was modeled with low-resistance state (LRS) and high-resistance state (HRS) of 10 k Ω and 100 k Ω , respectively. In the 3T2R array, VRH and VRL were set to 0.8 V and 0.5 V, respectively. The default INV was set to 2.0 V, which means the key element or threshold (k) in Eq. (1) is 0. The array size was fixed to 128 $\times$ 128. As the output of the 3T2R array is binary, no peripheral ADC circuit was necessary.

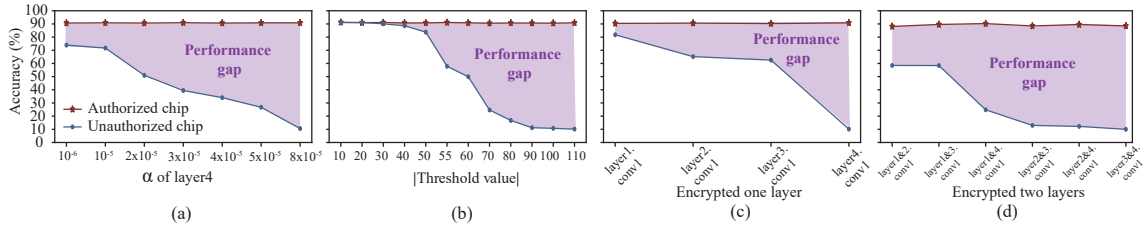


Fig. 7. IP protection effect on a ResNet model with (a) different scaling factors α ; (b) The absolute value of threshold; (c) Encryption position index and (d) dual-layer encryption effect.

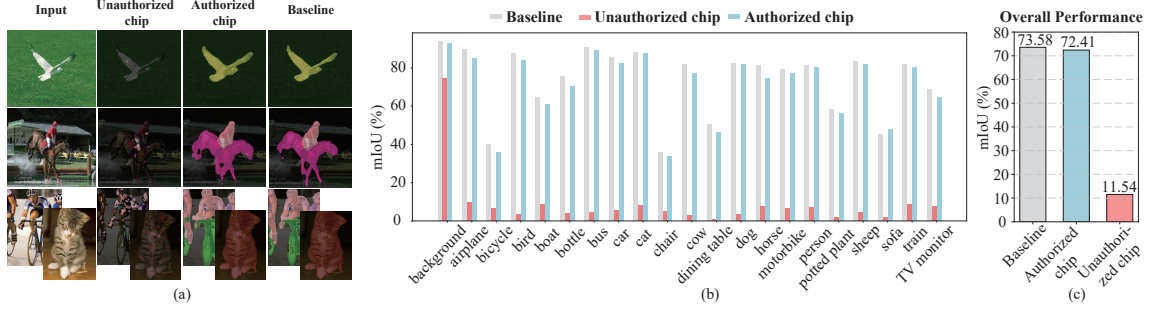


Fig. 8. IP protection effect on the semantic segmentation task. (a) Qualitative results; (b) Class-wise mean Intersection over Union (mIoU).

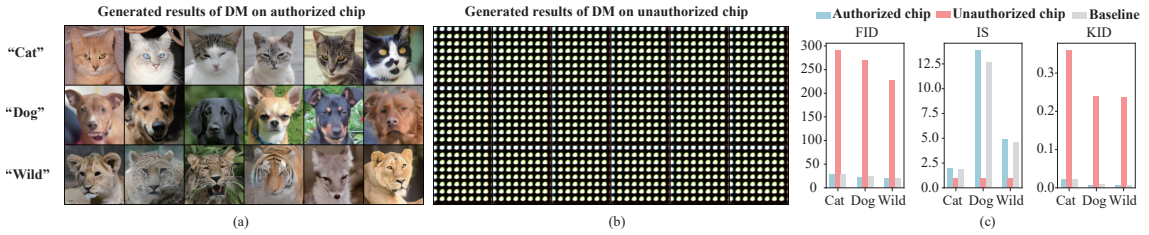


Fig. 9. IP protection effect on the image generation task. (a) Output from the authorized chip; (b) Output from the unauthorized chip; (c) Evaluation metrics, where Fréchet Inception Distance (FID) and Kernel Inception Distance (KID) are better when smaller, and Inception Score (IS) is better when larger.

B. Software IP Protection Evaluation

Object Classification. Fig. 7 analyzes the ResNet model under different hyperparameters, showing variations in the performance gap between authorized and unauthorized chips. Specifically, as illustrated in Fig. 7 (a), when α is very small (e.g., less than 10^{-5}), the performance gap is less than 20%. However, as α increases, the performance gap widens. Furthermore, the IP protection effect is proportional to the size of the secret key, or INV voltage, space. As shown in Fig. 7 (b), for the ResNet model, when the absolute value of the SL MAC threshold is less than 50 ($-50 < \text{"SL MAC threshold"} < 50$), and the corresponding INV voltage is between 1.6V and 2.3V (see Fig. 4 (c)), the performance gap is small, failing to achieve the desired protection effect. However, when the absolute value of the SL MAC threshold is 90 ($-90 < \text{"SL MAC threshold"} < 90$), with the corresponding INV voltage between 1.4V and 2.5V, the performance of the unauthorized chip approaches random guessing.

We also analyzed the impact of the impact of layer to be encrypted, as shown in Fig. 7 (c). When α is set to 0.0001 and the absolute value of the threshold is 110, encrypting deeper layers results in a larger performance gap. Fig. 7 (d) illustrates the effect of dual-layer encryption. Compared to single-layer encryption, dual-layer encryption significantly increases the performance gap.

Using these methods, we conducted generalization capability tests on both shallow MLPs and deep ViTs. The results

are presented in Fig. 6 (b). Our proposed contrastive training method significantly enhances the performance gap between authorized and unauthorized chips. Furthermore, compared to the baseline model running on a GPU, the model deployed on the authorized chip exhibits minimal performance degradation, with only a 1-2% decrease.

Semantic Segmentation. Fig. 8 demonstrates the effectiveness of IP protection in the segmentation task. For the model on unauthorized chips, most object-related pixels are misclassified. However, on the authorized chip, the segmentation results closely match the baseline segmentation performance (Fig. 8 (a)). Quantitatively, the segmentation ability on the unauthorized chip is very poor for almost all classes, except for the background. In contrast, on the authorized chip, the segmentation performance is nearly identical to that of the software baseline (Fig. 8 (b)). The overall mIoU of the model on the authorized chip reached 72.41%, while on the unauthorized chip it was only 11.54%, proving the effectiveness of the method.

Image Generation. Fig. 9 demonstrates the effectiveness of our method when applied to generative DiT models. Fig. 9 (a) shows that our model can generate high-quality image samples on the authorized chip, while in Fig. 9 (b), no meaningful images can be generated on the unauthorized chip. Fig. 9 (c) compares the image generation metrics between the authorized chip, the unauthorized chip, and the baseline. Across all three categories, the authorized chip (average FID: 23.31, IS:

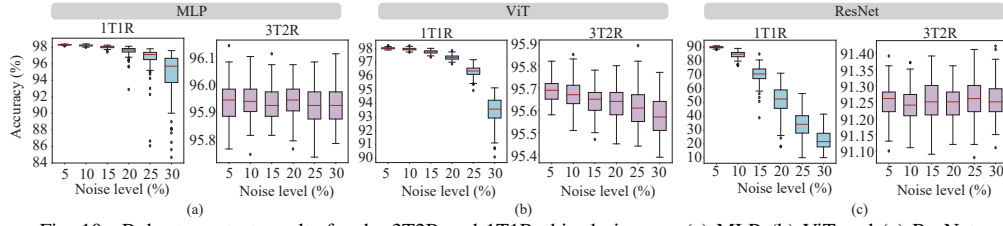


Fig. 10. Robustness test results for the 3T2R and 1T1R chip designs on (a) MLP, (b) ViT and (c) ResNet.

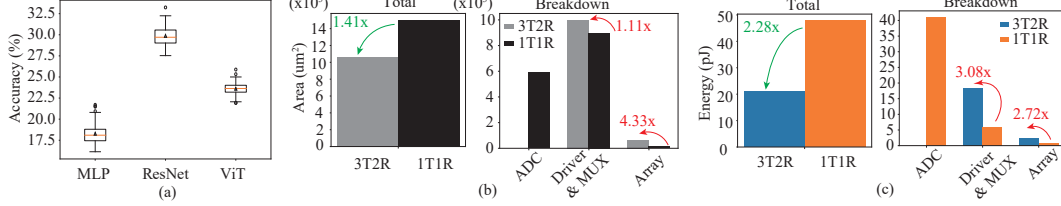


Fig. 11. Security and Area & Energy analysis. (a) Random guessing attack, and each model was tested over 100 trials. In the box, the orange line indicates the median, while the triangle denotes the mean; (b) Area and energy comparison between 3T2R and 1T1R.

6.89, KID: 0.012) achieves image quality comparable to the baseline (average FID: 24.58, IS: 6.40, KID: 0.012), while the unauthorized chip (average FID: 262.84, IS: 1.00, KID: 0.28) fails to generate any valid images.

C. Hardware Performance Evaluation

Robustness. Fig. 10 illustrates the robustness test results for the 3T2R and 1T1R chip designs under RRAM programming noise levels of 5%, 10%, 15%, 20%, 25%, and 30%, with each condition tested over 100 trials. As the noise level increases to 30%, the 3T2R chip’s accuracy declines by less than 1%, while the 1T1R chip’s accuracy drops significantly. Specifically, for a shallow model like MLP, as noise increases from 5% to 30%, the 3T2R chip exhibits almost no change in accuracy, whereas the 1T1R chip experiences a maximum decrease of 10%. However, for deeper models such as ViT or ResNet, the accuracy of the 1T1R chip gradually decreases with increasing noise, dropping by approximately 8% when noise reaches 30%. In contrast, the 3T2R chip remains very stable, showing a decline of less than 0.5%. This remarkable stability is primarily attributed to the INV inverter in the 3T2R design, which effectively filters out RRAM programming stochasticity compared to the 1T1R design [17].

Security. According to Kerckhoff’s Principle and Shannon’s Maxim [33], attackers can understand the protection method except for the secret keys. In this context, the secret key refers to the collection of INV voltages to 3T2R array. Assuming an array size of 128×128 , there are 128 INV voltages to be guessed. As shown in Fig. 4 (c), during normal operation, the range of INV can span from 1.2V to 2.5V. In practice, we assume INV takes discrete values at 0.1V intervals, giving each INV 14 possibilities. If an attacker attempts to use exhaustive search to find the exact key, the key space for an array of size 128×128 would be 14^{128} , which is an astronomical number. We also assume that the attacker can make random guesses; the performance of the model with 100 random trials remains extremely low, demonstrating the ineffectiveness of random guessing attacks (see Fig. 11 (a)).

Area & Energy. Fig. 11 (b) and (c) compare the area and energy consumption of 128×128 array chips using the

same software benchmarks. Compared to the 1T1R-based PIM accelerator, our 3T2R-based PIM accelerator achieves overall reductions in both area and energy consumption by $1.41 \times$ and $2.28 \times$, respectively. This is primarily due to the elimination of the need for ADCs in the 3T2R PIM. Specifically, although the array composed of the proposed 3T2R-based PIM occupies $4.33 \times$ more area and consumes $2.72 \times$ more energy than the 1T1R-based PIM [17], and the 3T2R design incurs $1.11 \times$ higher area and $3.08 \times$ higher energy costs for the driver and MUX components, the elimination of ADCs significantly compensates for the overhead introduced by the 3T2R-based PIM accelerator in terms of array and driver and MUX components.

V. CONCLUSION

RRAM-based PIM accelerators are promising for addressing energy consumption issues at the edge, but often overlook IP security and RRAM’s inherent stochasticity. This paper proposes a hardware-software co-design Guarder to address these challenges. We introduce a robust 3T2R cell design to mitigate RRAM stochasticity and reconfigure the array by adjusting the INV voltage, preventing attackers from exploiting stolen models. Additionally, a contrastive training method ensures that trained model parameters work only on authorized chips. We believe that the proposed methods enhance the security and robustness of RRAM in edge applications.

ACKNOWLEDGMENT

This research is supported by the National Key R&D Program of China (Grant No. 2022YFB3608300), National Natural Science Foundation of China under Grant 62488101, 62495104 and 62374181, Shenzhen Science and Technology Innovation Commission (Grant No. SGD20220530111405040), Beijing Natural Science Foundation (Grant No. Z210006), Hong Kong Research Grant Council (Grant Nos. 27209621, 17205922, 17212923), Joint Laboratory of Microelectronics (JLFS/E-601/24). This research was partially supported by ACCESS – AI Chip Center for Emerging Smart Systems, sponsored by InnoHK funding, Hong Kong SAR.

REFERENCES

- [1] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *ICLR*, 2021.
- [2] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4195–4205, 2023.
- [3] Pranav Rajpurkar, Awni Y Hannun, Masoumeh Haghighpanahi, Codie Bourn, and Andrew Y Ng. Cardiologist-level arrhythmia detection with convolutional neural networks. *arXiv preprint arXiv:1707.01836*, 2017.
- [4] AMD. Mercedes-benz accelerates innovation with amd technology. <https://www.amd.com/en/resources/case-studies/mercedes.html>.
- [5] AMD. Subaru leverages amd technology to enhance ai-powered automotive safety systems. <https://www.amd.com/en/resources/case-studies/subaru.html>.
- [6] Shuochao Yao, Yiran Zhao, Aston Zhang, Shaohan Hu, Huajie Shao, Chao Zhang, Lu Su, and Tarek Abdelzaher. Deep learning for the internet of things. *Computer*, 51(5):32–41, 2018.
- [7] Pratyush Mishra, Ryan Lehmkuhl, Akshayaram Srinivasan, Wenting Zheng, and Raluca Ada Popa. Delphi: A cryptographic inference system for neural networks. In *Proceedings of the 2020 Workshop on Privacy-Preserving Machine Learning in Practice*, pages 27–30, 2020.
- [8] Sourjya Roy, Shrihari Sridharan, Shubham Jain, and Anand Raghunathan. Txsim: Modeling training of deep neural networks on resistive crossbar systems. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 29(4):730–738, 2021.
- [9] Siddhartha Chhabra and Yan Solihin. i-nvmm: A secure non-volatile main memory system with incremental encryption. In *Proceedings of the 38th annual international symposium on Computer architecture*, pages 177–188, 2011.
- [10] Bin Gao, Bohan Lin, Yachuan Pang, Feng Xu, Yuyao Lu, Yen-Cheng Chiu, Zhengwu Liu, Jianshi Tang, Meng-Fan Chang, He Qian, et al. Concealable physically unclonable function chip with a memristor array. *Science advances*, 8(24):eabn7753, 2022.
- [11] Yi Cai, Xiaoming Chen, Lu Tian, Yu Wang, and Huazhong Yang. Enabling secure in-memory neural network computing by sparse fast gradient encryption. In *ICCAD*, pages 1–8, 2019.
- [12] Yi Cai, Xiaoming Chen, Lu Tian, Yu Wang, and Huazhong Yang. Enabling secure nvm-based in-memory neural network computing by sparse fast gradient encryption. *IEEE Transactions on Computers*, 69(11):1596–1610, 2020.
- [13] Ali Shafiee, Anirban Nag, Naveen Muralimanohar, Rajeev Balasubramanian, John Paul Strachan, Miao Hu, R. Stanley Williams, and Vivek Srikumar. Isaac: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars. In *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, pages 14–26, 2016.
- [14] Ping Chi, Shuangchen Li, Cong Xu, Tao Zhang, Jishen Zhao, Yongpan Liu, Yu Wang, and Yuan Xie. Prime: A novel processing-in-memory architecture for neural network computation in rram-based main memory. In *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, pages 27–39, 2016.
- [15] Zhongrui Wang, Saumil Joshi, Sergey E Savel'ev, Hao Jiang, Rivu Midya, Peng Lin, Miao Hu, Ning Ge, John Paul Strachan, Zhiyong Li, et al. Memristors with diffusive dynamics as synaptic emulators for neuromorphic computing. *Nature materials*, 16(1):101–108, 2017.
- [16] Linghao Song, Xuehai Qian, Hai Li, and Yiran Chen. Pipelayer: A pipelined rram-based accelerator for deep learning. In *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 541–552, 2017.
- [17] Peng Yao, Huaqiang Wu, Bin Gao, Jianshi Tang, Qingtian Zhang, Wenqiang Zhang, J Joshua Yang, and He Qian. Fully hardware-implemented memristor convolutional neural network. *Nature*, 577(7792):641–646, 2020.
- [18] Yulin Zhao, Jie Yu, Donglin Zhang, Qiao Hu, Xuanzhi Liu, Haijun Jiang, Qingting Ding, Zhongze Han, Jinhui Cheng, Wenjun Zhang, et al. A 0.02% accuracy loss voltage-mode parallel sensing scheme for rram-based xnor-net application. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 69(6):2697–2701, 2022.
- [19] Linfang Wang, Wang Ye, Chunmeng Dou, Xin Si, Xiaoxin Xu, Jing Liu, Dashan Shang, Jianfeng Gao, Feng Zhang, Yongpan Liu, et al. Efficient and robust nonvolatile computing-in-memory based on voltage division in 2t2r rram with input-dependent sensing control. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 68(5):1640–1644, 2021.
- [20] Miguel Angel Lastras-Montano and Kwang-Ting Cheng. Resistive random-access memory based on ratioed memristors. *Nature Electronics*, 1(8):466–472, 2018.
- [21] Qi Liu, Bin Gao, Peng Yao, Dong Wu, Junren Chen, Yachuan Pang, Wenqiang Zhang, Yan Liao, Cheng-Xin Xue, Wei-Hao Chen, Jianshi Tang, Yu Wang, Meng-Fan Chang, He Qian, and Huaqiang Wu. 33.2 a fully integrated analog rram based 78.4tops/w compute-in-memory chip with fully parallel mac computing. In *2020 IEEE International Solid-State Circuits Conference - (ISSCC)*, pages 500–502, 2020.
- [22] Minhui Zou, Zhenhua Zhu, Yi Cai, Junlong Zhou, Chengliang Wang, and Yu Wang. Security enhancement for rram computing system through obfuscating crossbar row connections. In *2020 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 466–471. IEEE, 2020.
- [23] Nvidia jetson nano developer kit. <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-nano/product-development/>. Accessed: 2023-10-20.
- [24] Weizhe Hua, Zhiru Zhang, and G Edward Suh. Reverse engineering convolutional neural networks through side-channel information leaks. In *Proceedings of the 55th Annual Design Automation Conference*, pages 1–6, 2018.
- [25] Lejla Batina, Shivam Bhasin, Dirmanto Jap, and Stjepan Picek. {CSI}{NN}: Reverse engineering of neural network architectures through electromagnetic side channel. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 515–532, 2019.
- [26] Qingli Guo, Jing Ye, Yue Gong, Yu Hu, and Xiaowei Li. Puf based pay-per-device scheme for ip protection of cnn model. In *2018 IEEE 27th Asian Test Symposium (ATS)*, pages 115–120. IEEE, 2018.
- [27] Ning Lin, Xiaoming Chen, Hang Lu, and Xiaowei Li. Chaotic weights: A novel approach to protect intellectual property of deep neural networks. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 40(7):1327–1339, 2021.
- [28] A. Skendzić et al. General data protection regulation — protection of personal data in an organisation. In *MIPRO*, 2018.
- [29] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [30] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [31] Alex Krizhevsky and Geoffrey Hinton. Learning multiple layers of features from tiny images. 2009.
- [32] Yunjeon Choi, Youngjun Uh, Jaehun Yoo, and Jung-Woo Ha. Stargan v2: Diverse image synthesis for multiple domains. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 8188–8197, 2020.
- [33] Claude E Shannon. Communication theory of secrecy systems. *The Bell system technical journal*, 1949.